Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

Лабораторная работа №3-4

«Функциональные возможности языка Python»

ДИСЦИПЛИНА: «Парадигмы и конструкции языков программирования»

Выполнил: студент гр. ИУ5-33Б _____ (Шестаков М.И.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю.Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

2026 г.

Цель: изучение возможностей функционального программирования в языке Python.

Задача: решить семь задач на использование генераторов, итераторов, декораторов, контекстных менеджеров и функций высшего порядка. Решения размещаются в пакете lab_python_fp, каждая задача — в отдельном файле.

Выполнение работы

1. Генератор field (field.py)

Генератор выдаёт значения одного поля или словари из нескольких полей, пропуская значения None.

```
def field(items, *args):
    assert len(args) > 0
    if len(args) == 1:
        key = args[0]
        for item in items:
            value = item.get(key)
            if value is not None:
                yield value
    else:
        for item in items:
            result = {key: item[key] for key in args if item.get(key) is not None}
            if result:
                yield result

if __name__ == '__main__':
    goods = [
        {'title': 'Ковер', 'price': 2000, 'color': 'green'},
        {'title': 'Диван для отдыха', 'price': 5300, 'color': 'black'},
        {'title': 'Стол', 'price': None, 'color': 'brown'},
        {'title': None, 'price': None, 'color': 'white'},
    ]
    print("field(goods, 'title'):")
    print(*field(goods, 'title'), sep=', ')
    print("field(goods, 'title', 'price'):")
    for item in field(goods, 'title', 'price'):
        print(item)
```

Листинг 1 – файл field.py

```
PS C:\Users\HONOR\prog\PCPL\Лаб.3-4\lab_python_fp> python field.py
field(goods, 'title'):
Ковер, Диван для отдыха, Стол
field(goods, 'title', 'price'):
{'title': 'Ковер', 'price': 2000}
{'title': 'Диван для отдыха', 'price': 5300}
{'title': 'Стол'}
PS C:\Users\HONOR\prog\PCPL\Лаб.3-4\lab_python_fp>
```

Рисунок 1 – результат работы field.py

2. Генератор случайных чисел (gen_random.py)

Генератор выдаёт заданное количество случайных чисел из диапазона с помощью random.randint.

```
import random

def gen_random(num_count, begin, end):
    for i in range(num_count):
        yield random.randint(begin, end)

if __name__ == '__main__':
    print('gen_random(5, 1, 3):')
    print(*gen_random(5, 1, 3), sep=', ')
    print('gen_random(10, -5, 5):')
    print(*gen_random(10, -5, 5), sep=', ')
```

Листинг 2 – файл gen_random.py

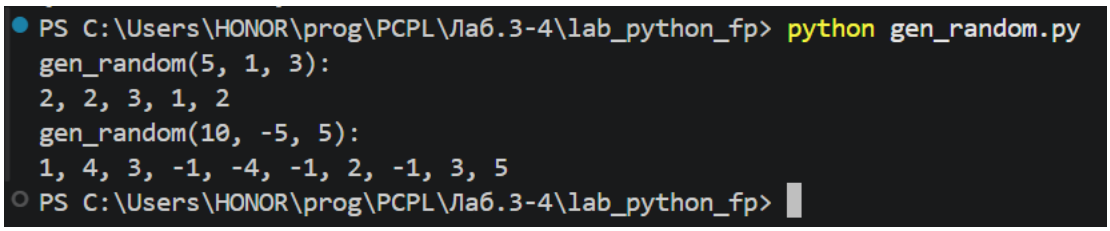


Рисунок 2 – результат работы gen_random.py

3. Итератор Unique (unique.py)

Итератор пропускает повторяющиеся элементы; параметр ignore_case передаётся через **kwargs и отключает учёт регистра строк.

```
try:
    from .gen_random import gen_random
except ImportError:
    from gen_random import gen_random

class Unique(object):
    def __init__(self, items, **kwargs):
        self.ignore_case = kwargs.get('ignore_case', False)
        self.items = iter(items)
        self.seen = set()

    def __next__(self):
        while True:
            item = next(self.items)
            key = item
            if self.ignore_case and isinstance(item, str):
                key = item.lower()
            if key not in self.seen:
                self.seen.add(key)
                return item

    def __iter__(self):
        return self

if __name__ == '__main__':
    data = [1, 1, 1, 1, 1, 2, 2, 2, 2, 2]
```

```
    print('Unique(data):', list(Unique(data)))

    data = gen_random(10, 1, 3)
    print('Unique(gen_random(10, 1, 3)):', list(Unique(data)))

    data = ['a', 'A', 'b', 'B', 'a', 'A', 'b', 'B']
    print('Unique(data):', list(Unique(data)))
    print('Unique(data, ignore_case=True):',
          list(Unique(data, ignore_case=True)))
```

Листинг 3 – файл unique.py

```
PS C:\Users\HONOR\prog\PCPL\Лаб.3-4\lab_python_fp> python unique.py
Unique(data): [1, 2]
Unique(gen_random(10, 1, 3)): [3, 2, 1]
Unique(data): ['a', 'A', 'b', 'B']
Unique(data, ignore_case=True): ['a', 'b']
PS C:\Users\HONOR\prog\PCPL\Лаб.3-4\lab_python_fp>
```

Рисунок 3 – результат работы unique.py

4. Сортировка по модулю (sort.py)

Массив сортируется по убыванию модуля функцией sorted без lambda-функции и с ней.

```
data = [4, -30, 30, 100, -100, 123, 1, 0, -1, -4]

if __name__ == '__main__':
    result = sorted(data, key=abs, reverse=True)
    print(result)

    result_with_lambda = sorted(data, key=lambda x: abs(x), reverse=True)
    print(result_with_lambda)
```

Листинг 4 – файл sort.py

```
PS C:\Users\HONOR\prog\PCPL\Лаб.3-4\lab_python_fp> python sort.py
[123, 100, -100, -30, 30, 4, -4, 1, -1, 0]
[123, 100, -100, -30, 30, 4, -4, 1, -1, 0]
PS C:\Users\HONOR\prog\PCPL\Лаб.3-4\lab_python_fp>
```

Рисунок 4 – результат работы sort.py

5. Декоратор print_result (print_result.py)

Декоратор печатает имя функции и её результат: список — в столбик, словарь — в виде «ключ = значение».

```
from functools import wraps

def print_result(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
```

```
        print(func.__name__)
        if isinstance(result, list):
            for item in result:
                print(item)
        elif isinstance(result, dict):
            for key, value in result.items():
                print(f'{key} = {value}')
        else:
            print(result)
        return result
    return wrapper

@print_result
def test_1():
    return 1

@print_result
def test_2():
    return 'iu5'

@print_result
def test_3():
    return {'a': 1, 'b': 2}

@print_result
def test_4():
    return [1, 2]

if __name__ == '__main__':
    print('!!!!!!!!')
    test_1()
    test_2()
    test_3()
    test_4()
```

Листинг 5 – файл print_result.py

```
PS C:\Users\HONOR\prog\PCPL\Лаб.3-4\lab_python_fp> python print_result.py
!!!!!!!!
test_1
1
test_2
iu5
test_3
a = 1
b = 2
test_4
1
2
PS C:\Users\HONOR\prog\PCPL\Лаб.3-4\lab_python_fp>
```

Рисунок 5 – результат работы print_result.py

6. Контекстные менеджеры (cm_timer.py)

cm_timer_1 реализован классом, cm_timer_2 — с помощью contextlib; оба выводят время выполнения блока with.

```
import time
from contextlib import contextmanager
```

```
class cm_timer_1:
    def __enter__(self):
        self.start = time.perf_counter()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        print(f'time: {time.perf_counter() - self.start:.3f}')
        return False

@contextmanager
def cm_timer_2():
    start = time.perf_counter()
    try:
        yield
    finally:
        print(f'time: {time.perf_counter() - start:.3f}')

if __name__ == '__main__':
    print('cm_timer_1, sleep(5.5):')
    with cm_timer_1():
        time.sleep(5.5)

    print('cm_timer_2, sleep(5.5):')
    with cm_timer_2():
        time.sleep(5.5)
```

Листинг 6 – файл cm_timer.py

```
PS C:\Users\HONOR\prog\PCPL\Лаб.3-4\lab_python_fp> python cm_timer.py
cm_timer_1, sleep(5.5):
time: 5.500
cm_timer_2, sleep(5.5):
time: 5.501
PS C:\Users\HONOR\prog\PCPL\Лаб.3-4\lab_python_fp>
```

Рисунок 6 – результат работы cm_timer.py

7. Обработка списка вакансий (process_data.py)

Функции f1–f4 по цепочке отбирают уникальные профессии, оставляют программистов (filter), добавляют «с опытом Python» (map) и зарплату (zip).

```
import json
import os
import sys

try:
    from .field import field
    from .gen_random import gen_random
    from .unique import Unique
    from .print_result import print_result
    from .cm_timer import cm_timer_1
except ImportError:
    from field import field
    from gen_random import gen_random
    from unique import Unique
    from print_result import print_result
    from cm_timer import cm_timer_1

if len(sys.argv) > 1:
    path = sys.argv[1]
else:
```

```
    script_dir = os.path.dirname(os.path.abspath(__file__))
    path = os.path.join(script_dir, 'data_light.json')

with open(path, encoding='utf-8') as f:
    data = json.load(f)

@print_result
def f1(arg):
    return sorted(Unique(field(arg, 'job-name'), ignore_case=True), key=str.lower)

@print_result
def f2(arg):
    return list(filter(lambda s: s.lower().startswith('программист'), arg))

@print_result
def f3(arg):
    return list(map(lambda s: s + ' с опытом Python', arg))

@print_result
def f4(arg):
    salaries = gen_random(len(arg), 100000, 200000)
    return [f'{p}, зарплата {s} руб.' for p, s in zip(arg, salaries)]

if __name__ == '__main__':
    with cm_timer_1():
        f4(f3(f2(f1(data))))
```

Листинг 7 – файл process_data.py

```
f2
Программист
Программист / Senior Developer
Программист 1С
Программист C#
Программист C++
Программист C++/C#/Java
Программист/ Junior Developer
Программист/ технический специалист
Программистр-разработчик  информационных систем
f3
Программист с опытом Python
Программист / Senior Developer с опытом Python
Программист 1С с опытом Python
Программист C# с опытом Python
Программист C++ с опытом Python
Программист C++/C#/Java с опытом Python
Программист/ Junior Developer с опытом Python
Программист/ технический специалист с опытом Python
Программистр-разработчик  информационных систем с опытом Python
f4
Программист с опытом Python, зарплата 113395 руб.
Программист / Senior Developer с опытом Python, зарплата 111269 руб.
Программист 1С с опытом Python, зарплата 137215 руб.
Программист C# с опытом Python, зарплата 119054 руб.
Программист C++ с опытом Python, зарплата 127166 руб.
Программист C++/C#/Java с опытом Python, зарплата 188635 руб.
Программист/ Junior Developer с опытом Python, зарплата 160409 руб.
Программист/ технический специалист с опытом Python, зарплата 176902 руб.
Программистр-разработчик  информационных систем с опытом Python, зарплата 152140 руб.
time: 0.100
PS C:\Users\HONOR\prog\PCPL\Лаб.3-4\lab_python_fp>
```

Рисунок 7 – результат работы process_data.py (окончание вывода)

Вывод: в ходе работы изучены функциональные возможности языка Python: генераторы, итераторы, lambda-функции, декораторы, контекстные менеджеры и функции filter, map, zip, sorted.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Быков, А. Ю. Решение задач на языках программирования Си и Си++ : методические указания / А. Ю. Быков. — Москва : МГТУ им. Н.Э. Баумана, 2017. — 248 с. — ISBN 978-5-7038-4577-6. — Текст : электронный // Лань : электронно-библиотечная система. — URL: https://e.lanbook.com/book/103505

2. Каширин, И. Ю. От Си к Си++ : учебное пособие / И. Ю. Каширин, В. С. Новичков. — 2-е изд., стер. — Москва : Горячая линия-Телеком, 2012. — 334 с. — ISBN 978-5-9912-0259-6. — Текст : электронный // Лань : электронно-библиотечная система. — URL: https://e.lanbook.com/book/5161

3. Быков А. Ю. Решение задач на языках программирования Си и Си++ : метод. указания к выполнению лаб. работ / Быков А. Ю. ; МГТУ им. Н. Э. Баумана. - М. : Изд-во МГТУ им. Н. Э. Баумана, 2017. - 244 с. : ил. - ISBN 978-5-7038-4577-6.

4. Иванова Г. С., Ничушкина Т. Н. Объектно-ориентированное программирование : учебник для вузов / Иванова Г. С., Ничушкина Т. Н. ; общ. ред. Иванова Г. С. - М. : Изд-во МГТУ Н.Э.Баумана, 2002. http://progbook.ru/technologiya-programmirovaniya/582-ivanova-tehnologiya-programmirovaniya.html

5. Иванова Г. С., Ничушкина Т. Н. Объектно-ориентированное программирование : учебник для вузов / Иванова Г. С., Ничушкина Т. Н. ; общ. ред. Иванова Г. С. - М. : Изд-во МГТУ им. Н. Э. Баумана, 2014. - 455 с. : ил. - Библиогр.: с. 450. - ISBN 978-5-7038-3921-8.

6. Подбельский В.В. Язык Си++: Учебное пособие. – М.: Финансы и статистика, 2003. http://progbook.ru/c/737-podbelskii-programmiovanie-na-yazyke-si.html.

7. Акулов О.А., Медведев Н.В. Информатика. Базовый курс. – М.: Омега-Л, 2006. http://razym.ru/naukaobraz/obrazov/151874-akulov-oa-medvedev-nv-informatika-bazovyy-kurs.html

8. Вычислительные методы и программирование. МГУ им. М.В. Ломоносова. ISSN 1726-3522. Журнал входит в 1-й уровень Белого списка научных журналов Минобрнауки России. https://num-meth.ru/index.php/journal/index

Дополнительные материалы

1. Иванова Г.С. Технология программирования: Учебник для вузов. – М.: Изд. МГТУ им. Ахо А.В., Хопкрофт Д.Э., Ульман Д.Д. Структуры данных и алгоритмы. – М., Вильямс, 2003. http://razym.ru/naukaobraz/obrazov/181547-aho-a-ulman-d-hopkroft-d-struktury-dannyh-i-algoritmy.html

2. Дейтел Х.М., Дейтел П.Дж. Как программировать на C++. – М.: Бином, 2001.
3. Т. Кормен Т., Лейзерсон Ч., Ривест Р. Алгоритмы: построение и анализ. – М. МЦНМО, 2005. http://rutracker.org/forum/viewtopic.php?t=533181
4. Джосьютис Н. C++ Стандартная библиотека для профессионалов. – СПб.: Питер, 2004. http://progbook.ru/c/178-dzhosyutis_c_standartnaya_biblioteka.html
5. Подбельский В.В. Стандартный Си++: Учебное пособие. – М.: Финансы и статистика, 2008.
6. Объектно-ориентированное программирование в C++: пер. с англ. / Лафоре Р. - 4-е изд. - СПб.: Питер, 2004. - 923 с. - (Классика computer science). - ISBN 5-94723-302-9.
7. Т. Кормен Т., Лейзерсон Ч., Ривест Р. Алгоритмы: построение и анализ. – М. МЦНМО, 2005.
8. Г. Шилдт. C++. Базовый курс, 3-е издание: Пер с англ. – М.: Издательский дом «Вильямс», 2011. – 624 с.
9. Павловская Т. А. C/C++. Программирование на языке высокого уровня: учебник для вузов / Павловская Т. А. - СПб.: Питер, 2003. - 460 с. - (Учебник для вузов). - ISBN 5-94723-568-4.
10. Бесплатные образовательные программы партнера (VK): https://education.vk.company/students